

Snowplow

A Java desktop game: MVC architecture, Swing UI, and classic object-oriented design patterns

Technical Documentation



Java • Swing • Object-Oriented Design • MVC • Javadoc

1. What It Does

A two-player, tick-driven simulation game in which players cooperate on a shared road network: one drives a snowplow to clear snow and ice, the other runs scheduled bus routes, and both can switch roles at runtime. Snow accumulates over time and progressively degrades lanes (thin snow → deep snow → ice → broken ice), bridges and tunnels behave differently under weather, and AI-driven traffic cars introduce collision risk. Money earned from clearing lanes and running routes is spent in an in-game store on new plow-head attachments and consumables.

The interesting part of this project, from an engineering standpoint, is less the gameplay loop itself and more the discipline behind how it was built: a fully decoupled domain model, validated headlessly through a custom test harness before a single pixel was drawn, then extended into a complete Swing application without touching the model's public API – covered in section 4, with the standout design decisions broken out in section 5.



2. Architecture

Strict Model-View-Controller separation across three packages, with a single Game object acting as the simulation clock. Every package has a narrow, well-defined responsibility and depends on the layer below it, never the other way around:

- `Game.tick()` → advance every vehicle → check pairwise lane collisions → advance weather – one deterministic step per frame, driven by GameController's game loop.

- GameController owns the simulation-facing logic (road/lane generation, input-to-action translation, scoring); ScreenController owns pure navigation between the menu, game, store and settings screens.

Package	Responsibility
src (Model)	Domain classes: vehicles (Snowplow, Bus, Car), the road network (Road, Lane, Intersection, Bridge, Tunnel, RoadNetwork), weather and lane-state hierarchy, player roles, the store, and the central Game clock.
controller	GameController (game loop, road/lane generation, collision & scoring logic), input mapping (KeyboardController, MouseController, InputMapper), ScreenController, AssetManager, SoundManager.
view	Swing screens and panels: main menu, game board and HUD, store, settings – rendered from model state, with no simulation logic of their own.

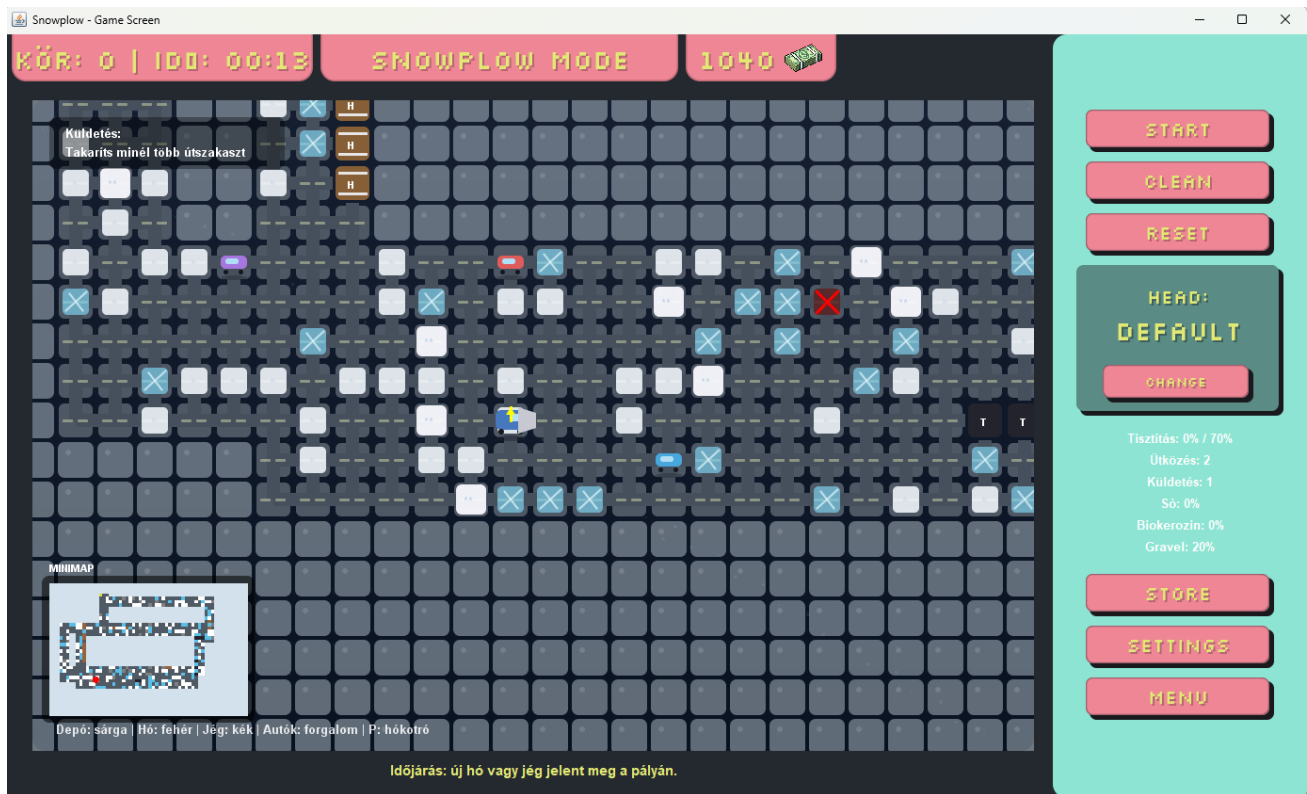


3. Design Patterns & Simulation Mechanics

- Strategy – the abstract Head class defines a cleaning strategy; six concrete heads (Thrower, Sweeper, Icebreaker, Salt Spreader, Gravel Spreader, Dragon) are purchased in the store and swapped on a Snowplow at runtime, each clearing lanes differently.
- Role – the abstract Role class (CleanerRole, BusdriverRole) separates what a player can currently do from the Player entity itself, so a player can hold and switch between responsibilities mid-game.
- Observer – ModelObservable / ModelObserver decouple the Swing views from the simulation model so the UI reacts to state changes without polling or tight coupling.
- Interface segregation – the Buyable interface is implemented by every purchasable entity (heads, consumables, vehicles), letting the Store treat them uniformly regardless of concrete type.
- Lane-state hierarchy – Clear, ThinSnow, DeepSnow, IceSheet, BrokenIce and Gravel form a polymorphic chain that models how a lane degrades under weather and recovers under player action.
- Graph-based road network – RoadNetwork, Node, Lane and Route model the map as a graph, supporting vehicle routing, AI-driven traffic cars, and pairwise collision detection each tick.

4. Development Process & Testing

- Staged delivery – the project moved through three explicit milestones: a skeleton defining public class contracts, a headless console prototype implementing and testing the full simulation, and finally this graphical version, which layers a Swing UI and controller logic on top of the validated model unchanged.
- Custom test harness – the prototype stage is exercised through an in-house arrange/act/assert command language (ArrangeContext, TestContext, TestSupport) driving console scenario tests, rather than a live UI.
- Documentation – every public class and method across all three packages is documented with Javadoc.
- Configurable input – InputMapper/InputAction decouple raw key codes from game actions, so key bindings can be remapped without touching controller logic.
- Centralized assets – AssetManager and SoundManager singletons preload sprites/fonts and manage background music and sound effects independently of the screens that use them.



5. Notable Engineering Highlights

These are the design decisions that go beyond a typical student game project and reflect deliberate architectural choices:

Runtime-swappable cleaning strategies

Rather than branching on plow type, Snowplow holds a reference to an abstract Head and delegates all cleaning behaviour to it. Buying a new head in the store simply calls `changeHead()`, so adding a seventh head type would require zero changes to Snowplow, Store, or the rendering code – a textbook open/closed-principle payoff from the Strategy pattern.

A model that never depended on its UI

The `src` package has no import on view or Swing anywhere. It was built and fully tested as a headless console application first; the graphical package was added afterward purely as a consumer of that model through `ModelObserver` notifications. This is why the project could grow from text-based to graphical without a rewrite – the dependency direction was correct from the start.

Graph-based road network with live traffic and collisions

The map is not a fixed image but a generated graph of nodes, lanes and roads (including bridges and tunnels with distinct weather rules). `GameController` generates this network, routes AI traffic cars across it, and runs pairwise

collision checks every tick – flagging lane accidents and temporarily disabling buses caught in them.

A progressive, physically-motivated weather model

Weather does not just toggle a boolean “snowing” flag: each lane progresses through a state chain (Clear → ThinSnow → DeepSnow → IceSheet → BrokenIce) driven by Weather.tick(), with bridges accumulating snow faster and tunnels staying immune – giving players a genuine reason to prioritize which lanes to clear first.

Dual-role players without duplicated state

A single Player can own both a CleanerRole and a BusdriverRole and switch its active role at runtime (checkRoleSwitch()), each with independent scoring, money and inventory – modelled through composition rather than subclassing Player per role, so adding a third role later would not touch the Player class at all.



6. Tech Stack

Category	Tools
Language	Java (JDK 8+)
UI / Rendering	Swing, Java2D, custom fonts (Silkscreen) and sprite assets
Audio	javax.sound (WAV background music &

	effects) via a dedicated SoundManager
Architecture	MVC, Strategy, Role, Observer, interface segregation
Testing	Custom console arrange/act/assert scenario harness (prototype stage)
Documentation	Javadoc across all public classes and methods
Build	javac / any Java IDE (IntelliJ IDEA, Eclipse, VS Code)

7. Skills Demonstrated

- System design and architecture – strict MVC layering with unidirectional dependencies, keeping the domain model fully independent of its user interface.
- Object-oriented design patterns – practical application of Strategy, Observer, Role composition and interface segregation to keep the codebase open to extension without modification.
- Graph algorithms – modelling a road network as a node/lane graph with routing, AI traffic, and per-tick collision detection.
- Event-driven / real-time simulation – a central tick-based game loop coordinating vehicles, weather, and scoring in lockstep.
- Desktop UI development – building a complete multi-screen Swing application (menu, HUD, store, settings) with custom rendering, fonts, and audio.
- Incremental software delivery – progressing from a class-contract skeleton to a tested headless prototype to a full application without breaking the model's public API.
- Test design – building a bespoke arrange/act/assert command-language test runner instead of relying on a live UI to validate simulation logic.
- Technical documentation – comprehensive Javadoc coverage and architecture write-ups aimed at onboarding a new reader quickly.

Source code available on GitHub